Introduction to computing for physicists

Your dearest, green

May 4, 2026

1 Introduction

Installation Create a file on your computer named "main.c", inside your are going to write the following:

```c
#include <stdio.h>

int main(void) {
  printf("Hello Seamen!\n");
  return 0;
}
```

now open a *terminal/shell* of your choice to *compile* and *run* the "main.c" file with the following:

```
# I am on Linux and use 'gcc' depending
# on your operating system you may need
# to use something else.
gcc -o main.c main
./main
```

if the phrase "Hello Seamen!" appeared on your terminal its good, you can move to the next paragraph. If nothing happened make sure everything has been properly written, if an error occurs *google* "How to install C compiler on Windows/Mac/Linux" or ask a trusted friend to do so for you. Repeat this process until the desired result has been reached.

Pre-requisites In order to move to the next chapter it is required to understand the behaviour of the following program without any external help:

```c
// 'int' is an abbreviation for integer
// '=' is an assignment, translate it with 'set to'
int factorial(int n) {
  int x = 1;
  loop:
  if (n > 0) {
    x = x * n;
    n = n - 1;
    goto loop;
  }
  else {
    return x;
  }
}
```

Can you guess the *return* value of x if $n = 5$? If you understood the code correctly it should be $x = 120$.

2 Program representation

2.1 Flow diagram representation

The rules for the representation of a flow diagram are the following, right after, a familiar diagram is shown.

- Circles indicate insertion and termination points, respectively named "start" and "end".
- Slanted parallelograms indicate insertion variables also known as **input** and termination variables also know as **output**.
- Rhombi are **conditionals** their exit flow is not fixed, its a function of their inner state. For our purposes the possible exit flows are *two*, one if the condition is satisfied and another one if it is not.
- Rectangles are **procedures**, their exit flow is fixed.

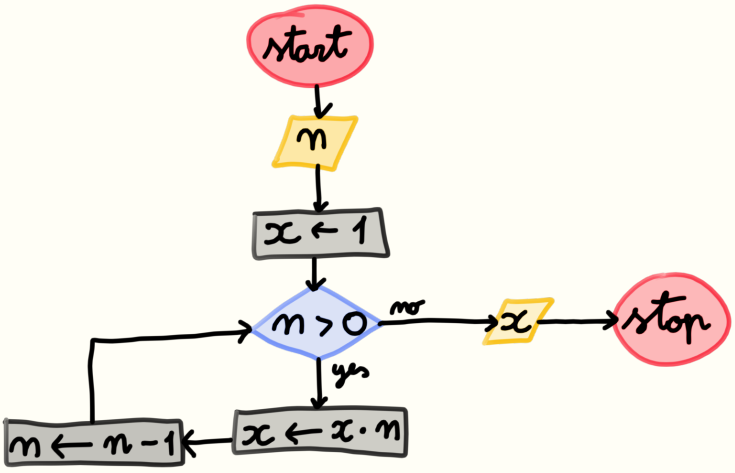


Figure 1: This is the flow diagram of the "factorial" program previously shown.

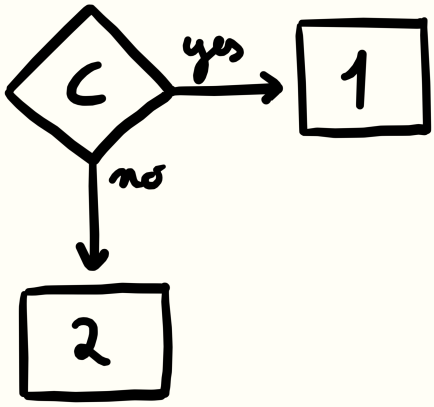
2.2 Procedural representation

The ordering given by arrows is now replaced There are no more arrows, each line is executed in order from top to bottom up until an *indirection* command such as "GOTO" which diverts the next execution to a special line marked with a "FLAG" statement. This is a typical scheme for how computers execute code.

```
START n
SETTO x 1
FLAG LOOP
IFSTART GREATERTHAN n 0
SETTO x PRODUCT x n
SETTO n SUBTRACTION n 1
GOTO LOOP
IFSTOP
ELSESTART
STOP x
ELSESTOP
```

2.3 Flow patterns

The flow of a program is a very important aspect to understand its behaviour, all useful programs have a source-sink aspect to them, all arrows begin from a source and they must all end in a sink at some point.



The *if-else* pattern conserves the tree structure of the program. If 1 and 2 are halting programs than the whole tree will halt.

```
if (C) {
  // 1
}
else {
  // 2
}
```

Figure 2: This is an **if-else** pattern, it starts from C.

The *if-elseif* pattern is a pattern extension of the *if-else* pattern. It conserves the tree structure of the program. If $1 \cdots n$ are halting programs than the whole tree will halt.

```
if (1) {
  // 1
}
else if (...) {
  // ...
}
else {
  // N
}
```

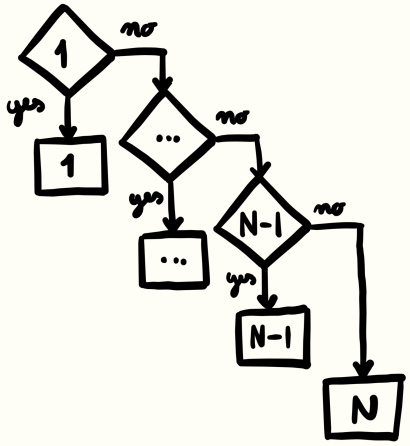


Figure 3: This is an **if-elseif** pattern, it starts from C.

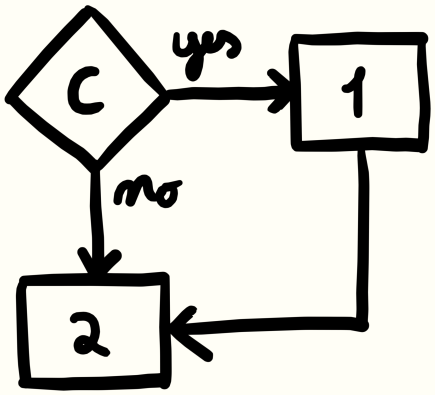


Figure 4: This is a free **if** pattern, it starts from C.

The *free if* pattern is a reduction of an *if-else* pattern. If $1 \otimes 2$ and 2 halts than the whole tree will halt

```
if (C) {
  // 1
}
// 2
```

The *while* pattern is guaranteed to halt if the condition C has an *upper-bound* check as a logical conjunction and is a function of a *strictly increasing* value (the same is true for a lower-bound with a stricly decreasing value).

```
while (C) {
  // B
}
```

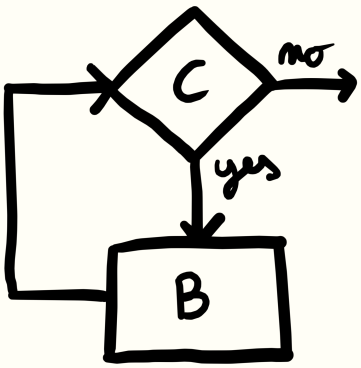


Figure 5: This is **while** pattern, it starts from C.

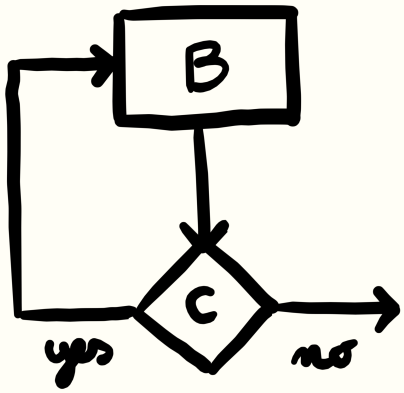


Figure 6: This is a **do-while** pattern, it starts from C.

The *do-while* pattern exhibits the same halting behaviour as the *while* pattern; the difference is that the *body* of the loop B will always resolve at least once.

```
do {
  // B
} while (C);
```

The *for* pattern exhibits the same halting behaviour as the *while* pattern; in addition if $C \otimes B = C \oplus B$ then the strictly increasing/decreasing value is controlled by IT.

```
for (IN; C; IT) {
  // B
}
```

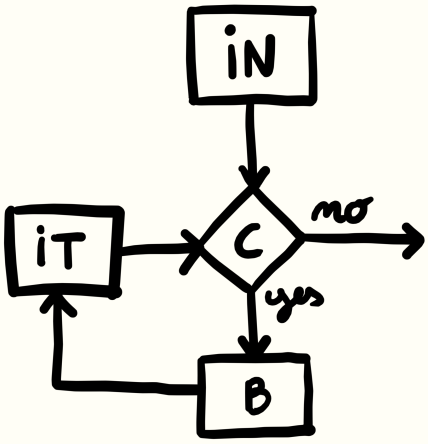


Figure 7: This is **for** pattern, it starts from IN.

Reductions

2.4 Exercises

For the following exercises read the instructions, draw the flow diagram, apply reductions if needed and implement the solution in C syntax.

Scale a row Given a row of floats $R[n]$ of size n and a factor fac, implement a program to scale each column entry by the same factor (row scaling)

```
void row_scale(int n, float R[n], float fac);
```

Add a row Given a two rows of floats $R0[n]$ and $R1[n]$ of size n and a factor fac, implement a program that adds together adjacent columns of $R0[n]$ and $R1[n]$ the latter being scaled by a factor of fac.

```
void row_add(int n, float R0[n], float R1[n], float fac);
```

Swap a row

Print a matrix Given a matrix $M[m][n]$ of size $m \times n$, implement a program that prints the matrix onto the screen.

```
void matrix_print(int m, int n, float M[m][n]);
```

keep in mind that to print something like

```
[1.00 6.67]
[3.14 6.96]
```

the code is the following

```
printf("[%.2f ", 1.00);
printf("%.2f]\n", 6.67);
printf("[%.2f ", 3.14);
printf("%.2f]\n", 6.69);
```

2.5 Solutions

Scale a row

```
void row_scale(int n, float R[n], float fac) {
  for (int j = 0; j < n; j = j + 1) {
    R[j] = fac * R[j];
  }
  return;
}
```

Add a row

```
void row_add(int n, float R0[n], float R1[n], float fac) {
  for (int j = 0; j < n; j = j + 1) {
    R0[j] = R0[j] + fac * R1[j];
  }
  return;
}
```

Swap a row

```
void row_swap(int n, float R0[n], float R1[n]) {
  float tmp = 0.0;
  for (int j = 0; j < n; j = j + 1) {
    tmp = R0[j];
    R0[j] = R1[j];
    R1[j] = tmp;
  }
  return;
}
```

Print a matrix

```
#include <stdio.h>

void matrix_print(int m, int n, float M[m][n]) {
  for (int i = 0; i < m; i = i + 1) {
    printf("[");
    for (int j = 0; j < n - 1; j = j + 1) {
      printf("%.2f ", M[i][j]);
    }
    printf("%.2f]\n", M[i][n-1]);
  }
  printf("\n");
  return;
}
```

2.6 Row reduction algorithm

The row reduction (also know as gaussian elimination) algorithm, is used to modify the form of a matrix into upper triangular without modifying its kernel. This algorithm is used to solve linear systems of equations.

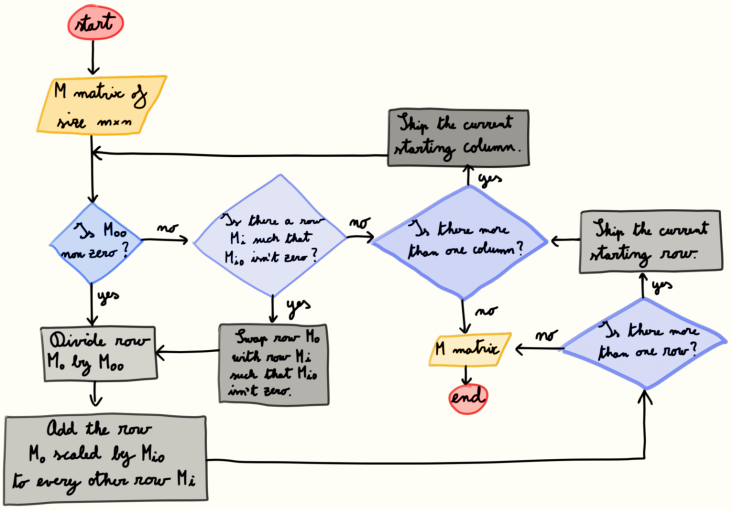


Figure 8: This is the (informal) flow diagram of the row-reduction algorithm for matrices.

Implementation

```c
int main(void) {
  float M[3][4] = {
    {0, 2, 3, 4},
    {1, 7, 7, 7},
    {9, 8, 7, 0}
  };
  row_reduction(3, 4, M);
  matrix_print(3, 4, M);
  return 0;
}
```

```c
void row_reduction(int m, int n, float M[m][n]) {
  int oi = 0;
  int oj = 0;
  main:
  if (M[oi][oj] != 0) {
    reduction:
    row_scale(n, M, 1/M[oi][oj]);
```

```
    for (int i = oi + 1; i < m; i = i + 1) {
      row_sub(n, M[i], M[oi], M[i][oj]);
    }
    if (oi < m) {
      oi = oi + 1;
      goto main;
    }
    else {
      return;
    }
  }
  else {
    if (exists_row_nonzero(m, n, M)) {
      row_swap_nearest_zero(m, n, M);
      goto reduction;
    }
    else {
      if (oj < n) {
        oj = oj + 1;
        goto main;
      }
      else {
        return;
      }
    }
  }
}
```

Optimizations After going through the hastle of translating the abstract algorithm represented by a diagram into the C language syntax and verifying its expected behaviour, one might be left unsatisfied and have some of these questions in mind:

- *The diagram is very simple, the code is very ugly. But it works great without any problems so its fine right?* It is until it isn't. Computer science tells us that there is no way of catching certain errors prior to running the program. Ergo write everything with the intention of making it easier to fix when it will break.

- *What has been just done can be applied to any language. Is the only advantage of C that of making it faster?* Although the fastest programs are written in C, that doesn't imply that writing any program in C will make it fast. The sole dictator of a program's speed is the hardware and the algorithm. The race between languages is a race where everyone gets the same car and are is only allowed to remove things from it. The C language doesn't care about its driver's well being (you), it just wants to win the race: no carpets, no seats, no airbags, no safety, no spedometer... If you are the best driver you will definitely win the race, otherwise you will just die (the program won't run and that will make you want to be not so nice towards yourself and other).

- *If my focus isn't performance is it worth learning C?* Yes it is. Following the previous answer, it has no other structure other than the absolutely necessary one.

In the implementation diagram 9 pay attention to the row swapping section. There is obviously some useless repetition carrying over from checking the possibility of doing a row swap at all, more importantly it introduces a hanging terminating condition (which shouldn't resolve but is still considered bad practice). It is better to remove it all together and adjust it so that the improved
10

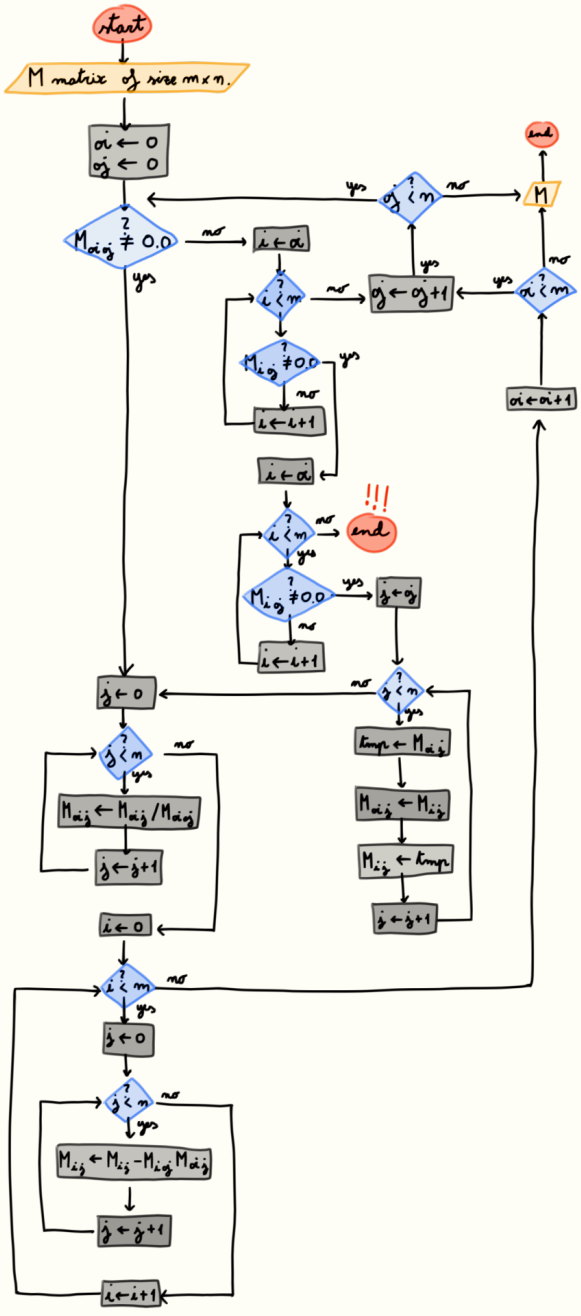


Figure 9: Formalized diagram of the previous row reduction algorithm implementation.

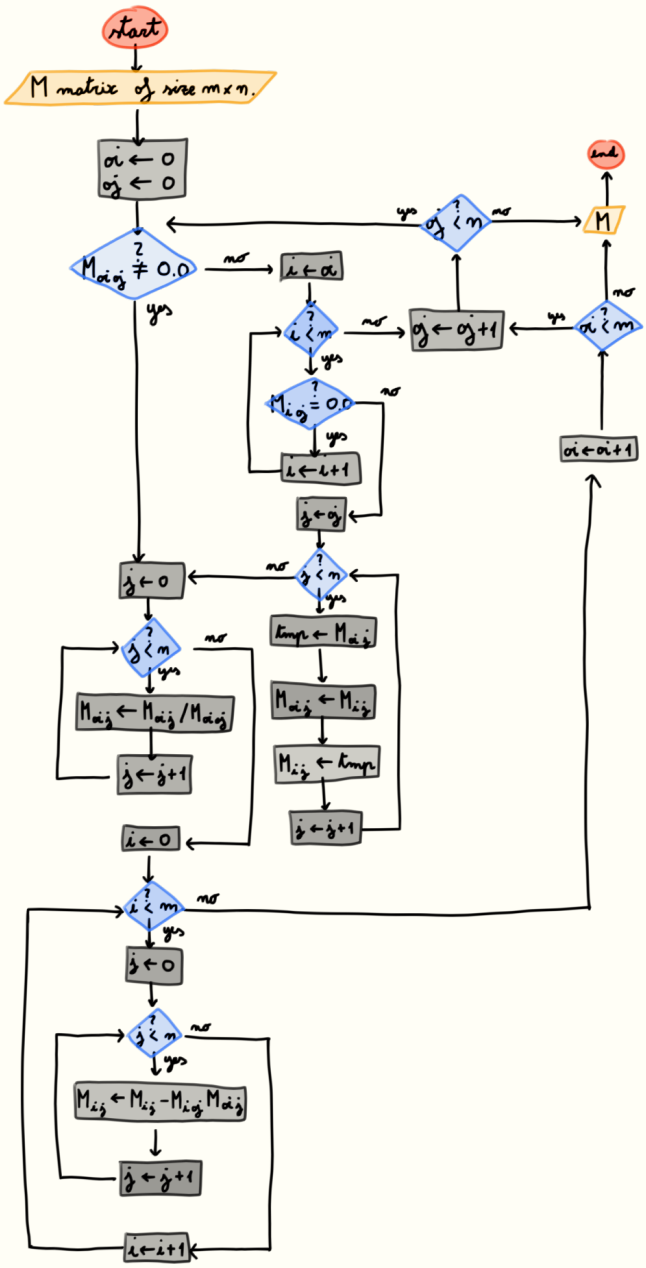


Figure 10: Formalized diagram of the previous row reduction algorithm implementation.

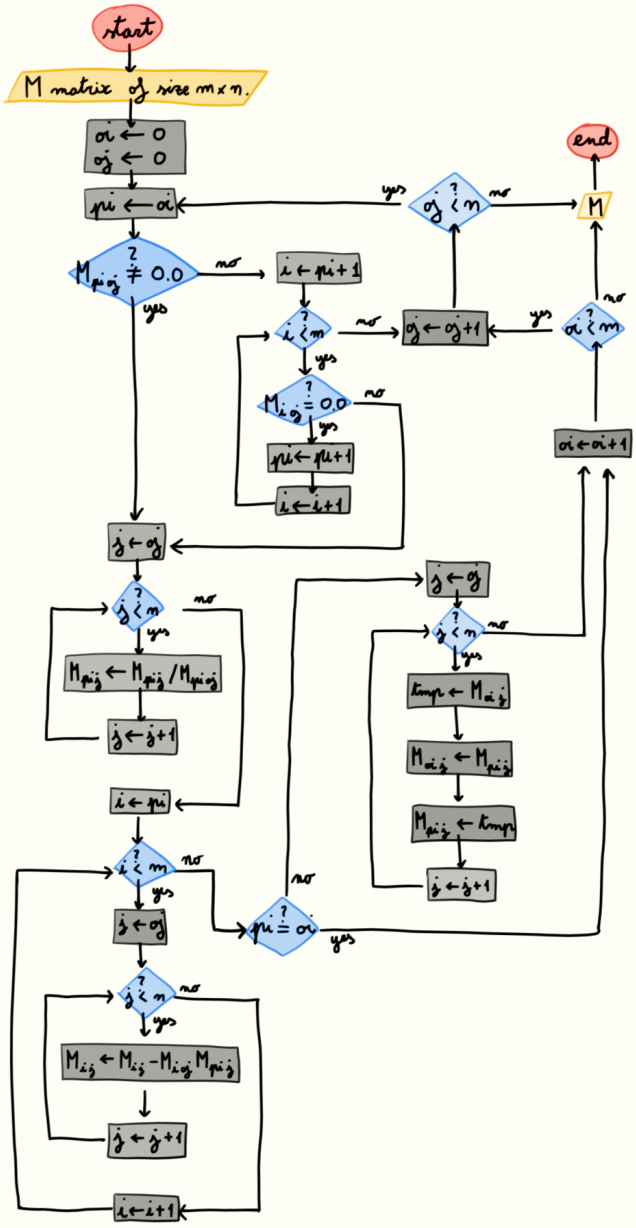


Figure 11: Formalized diagram of the previous row reduction algorithm implementation.

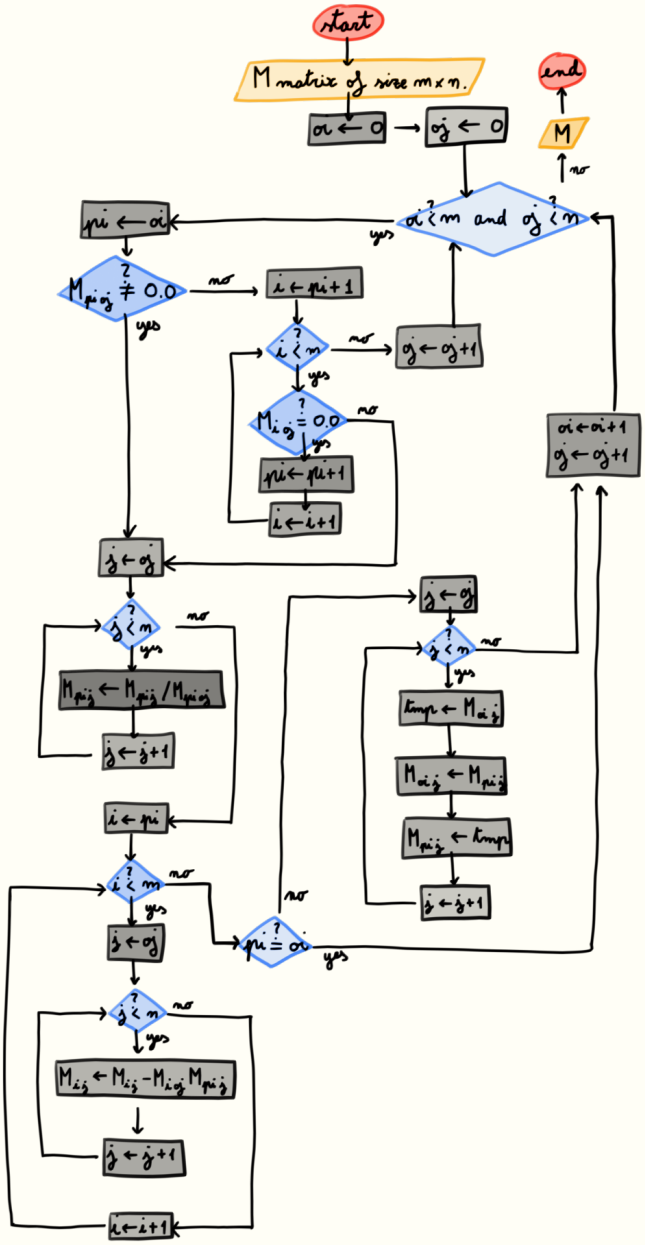


Figure 12: Formalized diagram of the previous row reduction algorithm implementation.

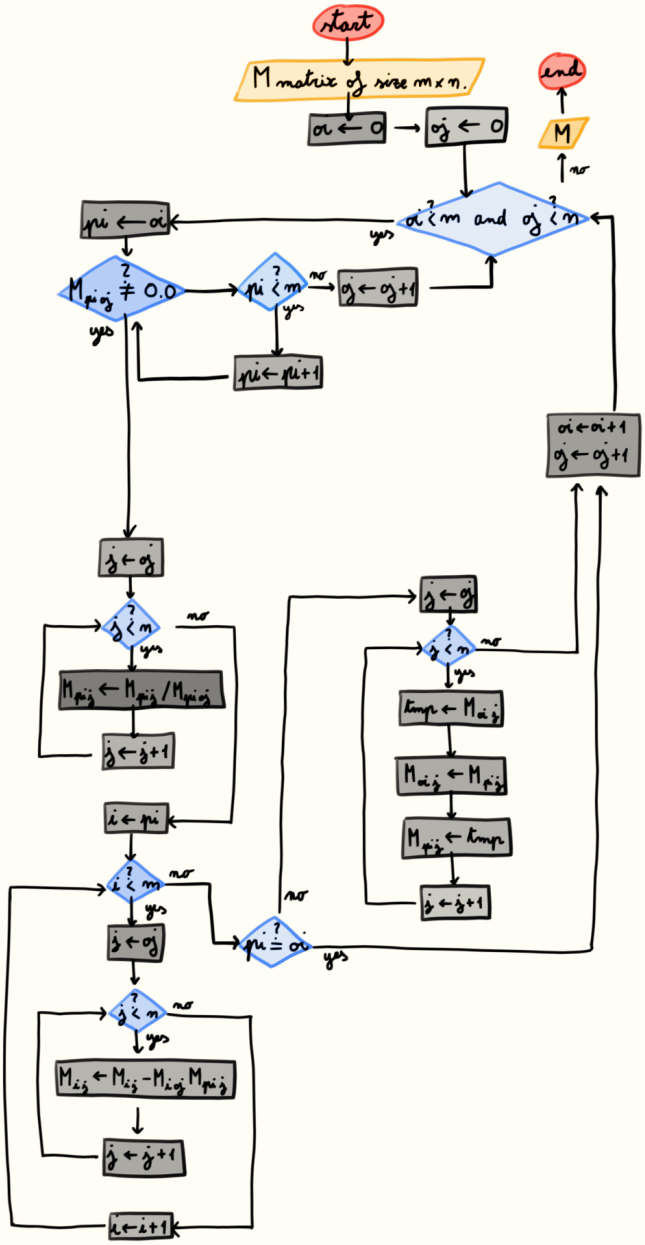


Figure 13: Formalized diagram of the previous row reduction algorithm implementation.

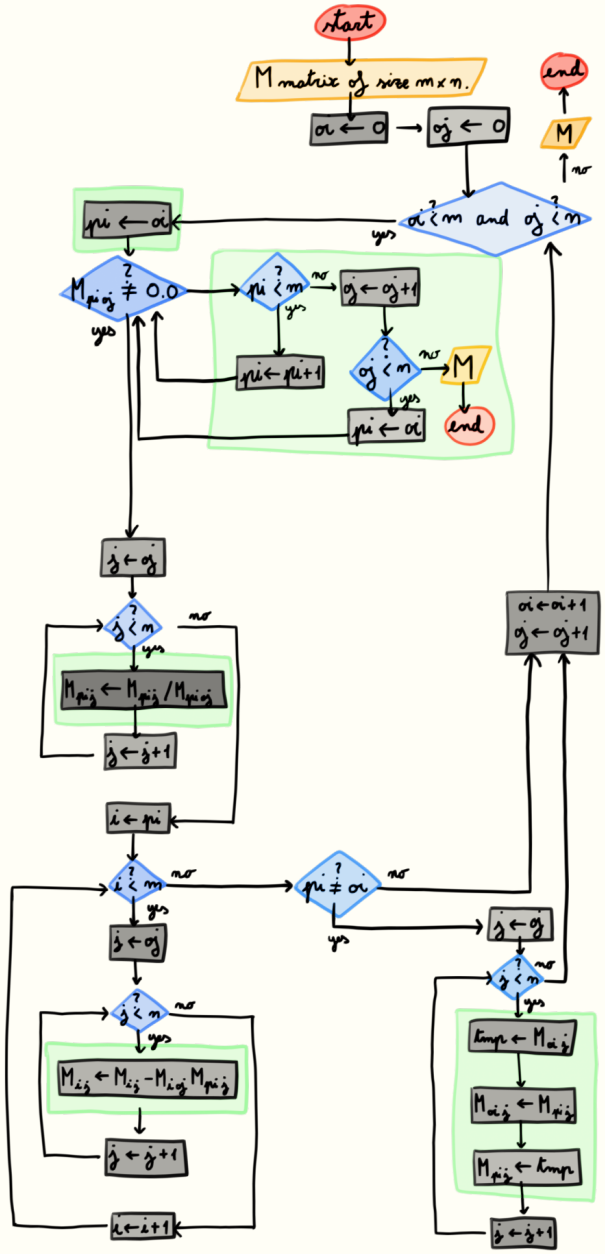


Figure 14: Formalized diagram of the previous row reduction algorithm implementation.

```c
void row_reduction(const int m, const int n, float M[m][n]) {
  for (
    int oi = 0, oj = 0, pi = oi;
    (oi < m && oj < n);
    {oi++; oj++; pi = oi;}
  ) {
    // Find pivot row.
    while (M[pi][oj] == 0.0) {
      if (pi < m - 1) {
        pi++;
      }
      else if (oj < n - 1) {
        pi = oi;
        oj++;
      }
      else {
        return;
      }
    }
    // Normalize pivot row.
    row_scale(n - oj, M[pi] + oj, 1/M[pi][oj]);
    // Clip offset column using pivot row as a mask.
    for (int i = pi + 1; i < m; i++) {
      row_add(n - oj, M[i] + oj, M[pi] + oj, -M[i][oj]);
    }
    // Swap pivot row with offset row.
    if (pi != oi) {
      row_swap(n - oj, M[oi] + oj, M[pi] + oj);
    }
  }
  return;
  /*
     [NOTES]
     ---------------------------------------
     Everything preceding the offset during
     matrix triangularization is such that:
     1) It doesn't change further steps.
     2) It isn't changed by further steps.
     ---------------------------------------
     therefore it is completely ignored by
     offsetting the matrices index.
     ---------------------------------------
  */
}
```

3 Data representation and ordering

3.1 Scope and pointers

FAQ

- What are pointers? An integer type.
- What do pointers do? I don't know, whatever you want your integers to do.
- What uses pointers? Referencing and de-referencing operators.
- What does (de-)referencing do? Allows you to access parallel scopes.
- What is a scope? A device mounted on top of a rifle.

Scope Up until this point we overlooked the idea of scope. In the C-language a scope is denoted by the curly braces enclosure.

```
{
  int a = 10;
  int a = 23;
  printf("%d\n", a);
  // error there are to many 'a'
}

{
  int a = 10;
  {
    int a = 23;
    printf("%d\n", a);
    // shows 23 on terminal
  }
  printf("%d\n", a);
  // shows 10 on terminal
}

{
  int a = 10;
  {
    int b = 67;
    printf("%d\n", a);
    // shows 10 on terminal
  }
  printf("%d\n", b);
  // error there is no such b
}
```

What happens when I call a function? Why does this happen?

```c
void funny(int a) {
  a = 67;
  return;
}

int main(void) {
  int a = 10;
  funny(a);
  printf("%d\n", a);
  // shows 10 on terminal
  return 0;
}
```

Lets observe the scope, this is what happens when we call a function

```c
funny:
{
  int a = 10;
  a = 67;
  goto _return;
}

{
  int a = 10;
  goto funny;
  return:
  printf("%d\n", a);
}
```

What about a pointer? What happens when I call a function? Why does this happen?

```c
void funny(int* a) {
  *a = 67;
  return;
}

int main(void) {
  int a = 10;
  funny(&a);
  printf("%d\n", a);
  // shows 10 on terminal
  return 0;
}
```

```c
funny:
```

```
{
  int* a = &a;
  *a = 67;
  goto _return;
}

{
  int a = 10;
  goto funny;
  return:
  printf("%d\n", 67);
}
```

Couldn't I just return 'a' instead? Yes but what if you want to change more than one variable?

3.2 Arrays

3.3 Integers

3.4 Decimals

3.5 Fixed point and floating point

4 Biblio

- "Informatica: Fondamenti Culturali e Tecnologici 1986" by *Michele Pellerey*
- "Introduction to Algorithms (third edition) 2009" by *Thomas H. Cormen*